

Plateforme Mastering Live Debix

Spécification Phase 1a — V4.1

Console numérique 8 canaux end-to-end
Intègre investigation critic V4 (6 workers, 854s)

Document de référence technique

2026-04-22 — rev V4.1

Table des matières

1	Ce qui change depuis V4	2
1.1	Bugs V4 corrigés en V4.1	2
1.2	Ce qui est confirmé et inchangé	2
2	Architecture cible (identique V3/V4)	3
3	Phasage V4.1	4
4	Phase 1a.1 MVP — squelette m4 final V4.1	5
4.1	Architecture	5
4.2	sof-imx8mp-tac5212-masterV41.m4 (top-level)	5
4.3	sof-imx8mp-tac5212-pipe-effects-capture.m4	5
4.4	sof-imx8mp-tac5212-pipe-effects-playback.m4	7
4.5	Board config Kconfig (inchangé V4)	9
5	Test T5 bit-perfect — limite découverte	9
6	Tests Phase 1a.1 MVP V4.1	9
7	Phase 1a.2 — stratégie tap HP_Monitor	9
8	Questions pour l’investigation V4.1	10
9	Livrables Phase 1a.1 MVP V4.1	11
10	Protocole	11

1 Ce qui change depuis V4

V4 a été soumis à une nouvelle investigation (6 workers : gemini-3-pro 102s, minimax 186s, glm-5.1 395s, qwen 435s, kimi 477s, claude-code 844s). **Verdict unanime : V4 NE COMPILE PAS tel quel.** V4.1 applique les corrections convergentes (claude-code a fait l'analyse la plus précise en lisant directement les patterns upstream).

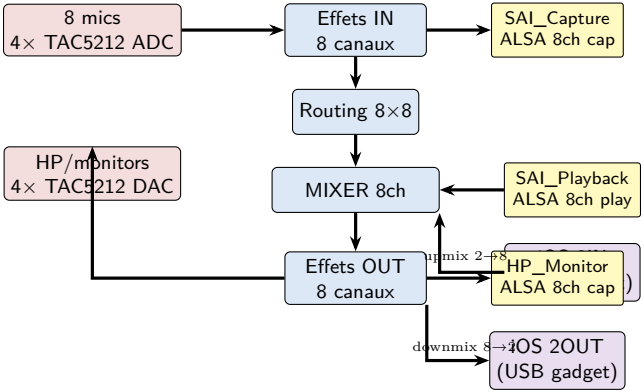
1.1 Bugs V4 corrigés en V4.1

#	Sévérité	Correction V4.1
B1	CRITICAL	<code>SOF_DRC_MAX_BLOB_SIZE</code> / <code>SOF_MULTIBAND_DRC_MAX_BLOB_SIZE</code> sont des <code>#define</code> C invisibles en m4. Remplacés par 1024 littéral (pattern upstream <code>pipe-drc-enabled-capture.m4:31</code>).
B2	CRITICAL	<code>W_PIPELINE(N_PCMC/N_PCMP, ...)</code> ajouté en V4 est redondant et buggué . Le scheduler widget est créé par <code>pipe-dai-capture.m4</code> via <code>DAI_ADD</code> sur le même pipeline_id. Ajouter un <code>W_PIPELINE</code> côté PCM crée un 2e scheduler → comportement indéfini. Supprimé en V4.1.
B3	CRITICAL	<code>multiband_drc_coef_default.m4</code> est en mode ACTIF (<code>emp_deemp=1</code>). Remplacé par <code>multiband_drc_coef_passthrough.m4</code> qui existe upstream et est un vrai bypass.
B4	HIGH	<code>drc_coef_default.m4</code> est aussi en mode actif (<code>enabled=1</code>) et aucun blob passthrough DRC n'existe upstream . Pire : même avec <code>enabled=0</code> , <code>drc_s32_default</code> applique un pre-delay de 256 frames (~5.3 ms). Conséquence : T5 sample-perfect impossible avec DRC dans la chaîne . V4.1 adopte 3 stratégies (voir §5).
B5	HIGH	kcontrol PGA "DEF_PGA_VOLUME" ne matche pas le nom <code>SectionControlMixer</code> généré par <code>C_CONTROLMIXER = "PIPELINE_ID Master Capture Volume"</code> . Corrigé.
B6	MEDIUM	<code>CONTROLMIXER</code> channel map : <code>VOLUME_CHANNEL_MAP</code> upstream (FL/FR) suffit — en mode linked, le PGA applique un gain unique aux 8 canaux. Pas besoin de 8 <code>KCONTROL_CHANNEL</code> explicites.
B7	MEDIUM	<code>W_PGA periods : capture = (2, DAI_PERIODS, ...)</code> , <code>playback = (DAI_PERIODS, 2, ...)</code> — cohérence upstream <code>pipe-volume-capture.m4:47</code> et <code>pipe-volume-playback.m4</code> .

1.2 Ce qui est confirmé et inchangé

- Architecture console 8ch end-to-end, les 4 PCMs, phasage
- Tous `SCHEDULE_TIME_DOMAIN_DMA` via `DAI_ADD` (pas de `TIMER`)
- Pas de cross-pipeline en Phase 1a.1 MVP
- Pattern `W_VENDORTUPLES` / `W_DATA` pour `DEF_PGA_CONF` validé par gemini-3-pro
- `CONFIG_COMP_MIXER=y` pour Phase 1a.3 (mixer limité 2 sources)
- Root cause V2.7 -110 : `PIPELINE_DEMUX_3` fantôme + guard `is_same_sched` `SCHEDULE_TIME_DOMAIN` mismatch. Pas "W_PIPELINE manquant" (erreur V3).

2 Architecture cible (identique V3/V4)

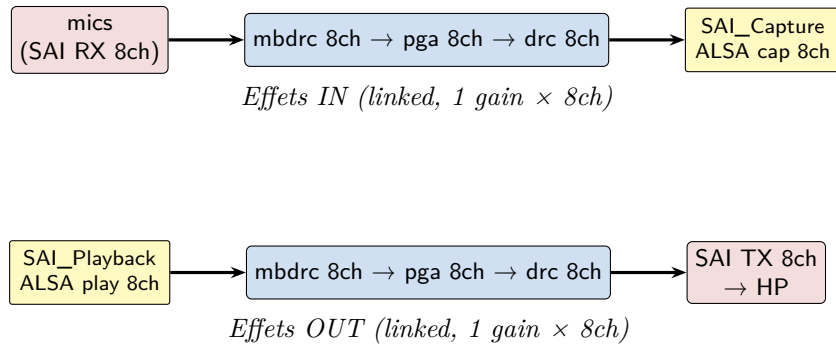


3 Phasage V4.1

Phase	Livrables	Objectif
1a.1 MVP	2 pipelines 8ch autonomes, effets linked (mbdrc+pga+drc), pas de cross-pipeline	Prouver pass-through + effets, trigger stable sans -110
1a.2	Tap HP_Monitor via option co-scheduled cross-pipeline same sched_comp	Capture post-effets OUT pour NPU
1a.3	Routing 8×8 + mixer 2 sources (mics routés + Playback) + injection SAI_Playback	iOS absorbé via routing slots 0-1, pas via mixer
1a.4	USB gadget iOS 2IN/2OUT	iOS connectable
2	USB ASIO gadget 8IN/8OUT	DAW sur host externe
3	Passage linked → per-channel	8 instances effets

4 Phase 1a.1 MVP — squelette m4 final V4.1

4.1 Architecture



4.2 sof-imx8mp-tac5212-masterV41.m4 (top-level)

```
#
# Topology V4.1 Phase 1a.1 MVP : console 8ch end-to-end, linked effects.
# 2 pipelines 8ch autonomes, SCHEDULE_TIME_DOMAIN_DMA via DAI_ADD.
# Le scheduler widget W_PIPELINE est créé par pipe-dai-{capture,playback}.m4.
#
include('utils.m4')
include('dai.m4')
include('pipeline.m4')
include('sai.m4')
include('pcm.m4')
include('buffer.m4')
include('common/tlv.m4')
include('sof/tokens.m4')
include('platform/imx/imx8.m4')

# PIPE 1 : SAI7 RX 8ch -> mbdrc -> pga -> drc -> SAI_Capture host
PIPELINE_PCM_ADD(sof/sof-imx8mp-tac5212-pipe-effects-capture.m4,
    1, 0, 8, s32le,
    1000, 0, 0,
    48000, 48000, 48000)

# PIPE 6 : SAI_Playback host -> mbdrc -> pga -> drc -> SAI7 TX 8ch
PIPELINE_PCM_ADD(sof/sof-imx8mp-tac5212-pipe-effects-playback.m4,
    6, 1, 8, s32le,
    1000, 0, 0,
    48000, 48000, 48000)

# DAI_ADD crée aussi le W_PIPELINE pour pipeline_id via pipe-dai-*.m4
DAI_ADD(sof/pipe-dai-capture.m4,
    1, SAI, 7, tac5212-hifi,
    PIPELINE_SINK_1, 2, s32le,
    1000, 0, 0, SCHEDULE_TIME_DOMAIN_DMA)

DAI_ADD(sof/pipe-dai-playback.m4,
    6, SAI, 7, tac5212-hifi,
    PIPELINE_SOURCE_6, 2, s32le,
    1000, 0, 0, SCHEDULE_TIME_DOMAIN_DMA)

PCM_CAPTURE_ADD(SAI_Capture, 0, PIPELINE_PCM_1)
PCM_PLAYBACK_ADD(SAI_Playback, 1, PIPELINE_PCM_6)

DAI_CONFIG(SAI, 7, 0, tac5212-hifi,
    SAI_CONFIG(DSP_A, SAI_CLOCK(mclk, 12288000, codec_mclk_in),
        SAI_CLOCK(bclk, 12288000, codec_consumer),
        SAI_CLOCK(fsyc, 48000, codec_consumer),
        SAI_TDM(8, 32, 255, 255),
        SAI_CONFIG_DATA(SAI, 7, 0)))
```

4.3 sof-imx8mp-tac5212-pipe-effects-capture.m4

```
# Capture 8ch : DAI SAI RX -> B3 -> MBDR -> B2 -> PGA -> B1 -> DRC -> B0 -> HOST
```

```

# Pattern direct : pipe-volume-capture.m4 + pipe-drc-enabled-capture.m4 + mbdrc.

include('utils.m4')
include('buffer.m4')
include('pcm.m4')
include('dai.m4')
include('bytecontrol.m4')
include('mixercontrol.m4')
include('pipeline.m4')
include('multiband_drc.m4')
include('pga.m4')
include('drc.m4')

# PGA volume kcontrol (nom complet = "<PIPELINE_ID> Master Capture Volume")
C_CONTROLMIXER(Master Capture Volume, PIPELINE_ID,
    CONTROLMIXER_OPS(volsw,
        256 binds the mixer control to volume get/put handlers, 256, 256),
    CONTROLMIXER_MAX(, 32),
    false,
    CONTROLMIXER_TLV(TLV 32 steps from -64dB to 0dB for 2dB, vtlv_m64s2),
    Channel register and shift for Front Left/Right,
    VOLUME_CHANNEL_MAP)

# PGA vendor tuples (pattern pipe-volume-capture.m4)
define(DEF_PGA_TOKENS, concat('pga_tokens_', PIPELINE_ID))
define(DEF_PGA_CONF, concat('pga_conf_', PIPELINE_ID))
W_VENDORTUPLES(DEF_PGA_TOKENS, sof_volume_tokens,
    LIST(' ', 'SOF_TKN_VOLUME_RAMP_STEP_TYPE "0"',
        , 'SOF_TKN_VOLUME_RAMP_STEP_MS "250"'))
W_DATA(DEF_PGA_CONF, DEF_PGA_TOKENS)

# MBDRC byte control + blob BYPASS (multiband_drc_coef_passthrough.m4 existe upstream)
define(MULTIBAND_DRC_priv, concat('mbdrc_bytes_', PIPELINE_ID))
define(MY_MULTIBAND_DRC_CTRL, concat('mbdrc_control_', PIPELINE_ID))
include('multiband_drc_coef_passthrough.m4')
C_CONTROLBYTES(MY_MULTIBAND_DRC_CTRL, PIPELINE_ID,
    CONTROLBYTES_OPS(bytes,
        258 binds the control to bytes get/put handlers, 258, 258),
    CONTROLBYTES_EXTOPS(
        258 binds the control to bytes get/put handlers, 258, 258),
    , ,
    CONTROLBYTES_MAX(, 1024),
    ,
    MULTIBAND_DRC_priv)

# DRC byte control + blob (default ACTIF : pour T5 bit-perfect, voir section T5)
define(DRC_priv, concat('drc_bytes_', PIPELINE_ID))
define(MY_DRC_CTRL, concat('drc_control_', PIPELINE_ID))
include('drc_coef_default.m4')
C_CONTROLBYTES(MY_DRC_CTRL, PIPELINE_ID,
    CONTROLBYTES_OPS(bytes,
        258 binds the control to bytes get/put handlers, 258, 258),
    CONTROLBYTES_EXTOPS(
        258 binds the control to bytes get/put handlers, 258, 258),
    , ,
    CONTROLBYTES_MAX(, 1024),
    ,
    DRC_priv)

# Host PCM endpoint
W_PCM_CAPTURE(PCM_ID, SAI Capture, 0, 2, SCHEDULE_CORE)

# Widgets : mbdrc -> pga -> drc
W_MULTIBAND_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
    LIST(' ', "MY_MULTIBAND_DRC_CTRL"))
W_PGA(0, PIPELINE_FORMAT, 2, DAI_PERIODS, DEF_PGA_CONF, SCHEDULE_CORE,
    LIST(' ', "PIPELINE_ID Master Capture Volume"))
W_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
    LIST(' ', "MY_DRC_CTRL"))

# Buffers : B0 host side, B3 DAI side
W_BUFFER(0, COMP_BUFFER_SIZE(2,
    COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_HOST_MEM_CAP)
W_BUFFER(1, COMP_BUFFER_SIZE(2,

```

```

    COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_COMP_MEM_CAP)
W_BUFFER(2, COMP_BUFFER_SIZE(2,
    COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_COMP_MEM_CAP)
W_BUFFER(3, COMP_BUFFER_SIZE(DAI_PERIODS,
    COMP_SAMPLE_SIZE(DAI_FORMAT), PIPELINE_CHANNELS,
    COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
    PLATFORM_DAI_MEM_CAP)

# PAS DE W_PIPELINE ici (bug V4 B2).
# Scheduler widget fourni par pipe-dai-capture.m4 lors de DAI_ADD.

# Graph DAPM : capture = flux DAI -> HOST, dapm(sink, source)
P_GRAPH(pipe-effects-capture, PIPELINE_ID,
    LIST(
        'dapm(N_PCMC(PCM_ID), N_BUFFER(0))',
        'dapm(N_BUFFER(0), N_DRC(0))',
        'dapm(N_DRC(0), N_BUFFER(1))',
        'dapm(N_BUFFER(1), N_PGA(0))',
        'dapm(N_PGA(0), N_BUFFER(2))',
        'dapm(N_BUFFER(2), N_MULTIBAND_DRC(0))',
        'dapm(N_MULTIBAND_DRC(0), N_BUFFER(3))'))

# Exports pour DAI_ADD
indir('define', concat('PIPELINE_SINK_', PIPELINE_ID), N_BUFFER(3))
indir('define', concat('PIPELINE_PCM_', PIPELINE_ID), SAI Capture PCM_ID)

PCM_CAPABILITIES(SAI Capture PCM_ID, CAPABILITY_FORMAT_NAME(PIPELINE_FORMAT),
    PCM_MIN_RATE, PCM_MAX_RATE, 2, PIPELINE_CHANNELS, 2, 16,
    192, 16384, 65536, 65536)

undefine('MY_MULTIBAND_DRC_CTRL') undefine('MULTIBAND_DRC_priv')
undefine('MY_DRC_CTRL') undefine('DRC_priv')
undefine('DEF_PGA_TOKENS') undefine('DEF_PGA_CONF')

```

4.4 sof-imx8mp-tac5212-pipe-effects-playback.m4

```

# Playback 8ch : HOST -> B0 -> MBDRD -> B1 -> PGA -> B2 -> DRC -> B3 -> DAI SAI TX
# Pattern direct : pipe-multiband-drc-playback.m4 + pga + drc chainage.

include('utils.m4')
include('buffer.m4')
include('pcm.m4')
include('dai.m4')
include('bytecontrol.m4')
include('mixercontrol.m4')
include('pipeline.m4')
include('multiband_drc.m4')
include('pga.m4')
include('drc.m4')

C_CONTROLMIXER(Master Playback Volume, PIPELINE_ID,
    CONTROLMIXER_OPS(volsw,
        256 binds the mixer control to volume get/put handlers, 256, 256),
    CONTROLMIXER_MAX(, 32),
    false,
    CONTROLMIXER_TLV(TLV 32 steps from -64dB to 0dB for 2dB, vtlv_m64s2),
    Channel register and shift for Front Left/Right,
    VOLUME_CHANNEL_MAP)

define(DEF_PGA_TOKENS, concat('pga_tokens_', PIPELINE_ID))
define(DEF_PGA_CONF, concat('pga_conf_', PIPELINE_ID))
W_VENDORTUPLES(DEF_PGA_TOKENS, sof_volume_tokens,
    LIST(
        'SOF_TKN_VOLUME_RAMP_STEP_TYPE "2"',
        'SOF_TKN_VOLUME_RAMP_STEP_MS "20"'))
W_DATA(DEF_PGA_CONF, DEF_PGA_TOKENS)

# MBDRD bypass (blob passthrough upstream)
define(MULTIBAND_DRC_priv, concat('mbdrc_bytes_', PIPELINE_ID))
define(MY_MULTIBAND_DRC_CTRL, concat('mbdrc_control_', PIPELINE_ID))
include('multiband_drc_coef_passthrough.m4')

```

```

C_CONTROLBYTES(MY_MULTIBAND_DRC_CTRL, PIPELINE_ID,
CONTROLBYTES_OPS(bytes,
    258 binds the control to bytes get/put handlers, 258, 258),
CONTROLBYTES_EXTOPS(
    258 binds the control to bytes get/put handlers, 258, 258),
, , , CONTROLBYTES_MAX(, 1024), ,
MULTIBAND_DRC_priv)

# DRC default blob (actif, voir section T5)
define(DRC_priv, concat('drc_bytes_', PIPELINE_ID))
define(MY_DRC_CTRL, concat('drc_control_', PIPELINE_ID))
include('drc_coef_default.m4')
C_CONTROLBYTES(MY_DRC_CTRL, PIPELINE_ID,
CONTROLBYTES_OPS(bytes,
    258 binds the control to bytes get/put handlers, 258, 258),
CONTROLBYTES_EXTOPS(
    258 binds the control to bytes get/put handlers, 258, 258),
, , , CONTROLBYTES_MAX(, 1024), ,
DRC_priv)

# Host playback endpoint
W_PCM_PLAYBACK(PCM_ID, SAI Playback, 2, 0, SCHEDULE_CORE)

# Widgets : mbdrc -> pga (W_PGA periods inversés pour playback) -> drc
W_MULTIBAND_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
LIST(' ', "MY_MULTIBAND_DRC_CTRL"))
W_PGA(0, PIPELINE_FORMAT, DAI_PERIODS, 2, DEF_PGA_CONF, SCHEDULE_CORE,
LIST(' ', "PIPELINE_ID Master Playback Volume"))
W_DRC(0, PIPELINE_FORMAT, 2, 2, SCHEDULE_CORE,
LIST(' ', "MY_DRC_CTRL"))

# Buffers : B0 host side, B3 DAI side
W_BUFFER(0, COMP_BUFFER_SIZE(2,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_HOST_MEM_CAP)
W_BUFFER(1, COMP_BUFFER_SIZE(2,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_COMP_MEM_CAP)
W_BUFFER(2, COMP_BUFFER_SIZE(2,
COMP_SAMPLE_SIZE(PIPELINE_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_COMP_MEM_CAP)
W_BUFFER(3, COMP_BUFFER_SIZE(DAI_PERIODS,
COMP_SAMPLE_SIZE(DAI_FORMAT), PIPELINE_CHANNELS,
COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD)),
PLATFORM_DAI_MEM_CAP)

# PAS DE W_PIPELINE ici (scheduler fourni par pipe-dai-playback.m4)

# Graph DAPM : playback = flux HOST -> DAI, dapm(sink, source)
P_GRAPH(pipe-effects-playback, PIPELINE_ID,
LIST(' ',
'dapm(N_BUFFER(0), N_PCMP(PCM_ID))',
'dapm(N_MULTIBAND_DRC(0), N_BUFFER(0))',
'dapm(N_BUFFER(1), N_MULTIBAND_DRC(0))',
'dapm(N_PGA(0), N_BUFFER(1))',
'dapm(N_BUFFER(2), N_PGA(0))',
'dapm(N_DRC(0), N_BUFFER(2))',
'dapm(N_BUFFER(3), N_DRC(0))'))

# Exports pour DAI_ADD
indir('define', concat('PIPELINE_SOURCE_', PIPELINE_ID), N_BUFFER(3))
indir('define', concat('PIPELINE_PCM_', PIPELINE_ID), SAI Playback PCM_ID)

PCM_CAPABILITIES(SAI Playback PCM_ID, CAPABILITY_FORMAT_NAME(PIPELINE_FORMAT),
PCM_MIN_RATE, PCM_MAX_RATE, 2, PIPELINE_CHANNELS, 2, 16,
192, 16384, 65536, 65536)

undefine('MY_MULTIBAND_DRC_CTRL') undefine('MULTIBAND_DRC_priv')
undefine('MY_DRC_CTRL') undefine('DRC_priv')
undefine('DEF_PGA_TOKENS') undefine('DEF_PGA_CONF')

```


4.5 Board config Kconfig (inchangé V4)

```
CONFIG_IMX8M=y
CONFIG_HAVE_AGENT=n
CONFIG_FORMAT_CONVERT_HIFI3=n
CONFIG_KPB_FORCE_COPY_TYPE_NORMAL=n

# Phase 1a.1 MVP
CONFIG_COMP_DRC=y
CONFIG_COMP_VOLUME=y
CONFIG_COMP_CROSSOVER=y
CONFIG_COMP_MULTIBAND_DRC=y
CONFIG_COMP_MUX=y

# Ajouté V4 pour Phase 1a.3
CONFIG_COMP_MIXER=y
```

5 Test T5 bit-perfect — limite découverte

L'investigation V4 (claude-code) a relevé un point important : même avec `enabled=0` côté DRC, la fonction `drc_s32_default` (`drc_generic.c:744-750`) applique un **pre-delay de 256 frames** (~5.3 ms @48k). Le null test sample-perfect **n'est donc pas atteignable avec DRC dans la chaîne**.

Trois stratégies T5 :

1. **T5a compensé** : loopback HW, null test avec compensation de latence (~256 frames DRC + buffers). Tolérance ± 1 LSB. Faisable, pragmatique.
2. **T5b sans DRC** : topology variante `masterV41-noDRC` qui skip le widget DRC dans la chaîne. Null test sample-perfect. Utilisé uniquement pour la gate V4.1.
3. **T5c blob patché** : créer localement un `drc_coef_zerolatency.m4` avec `enabled=0` ET `pre_delay_time=0`. Forcerait `drc_default_pass = audio_stream_copy()`. Pratique long terme mais patch custom.

Choix V4.1 : T5a en premier passage (pragmatique), T5c comme option long terme si T5a peine à converger.

6 Tests Phase 1a.1 MVP V4.1

1. **T0 Compile** : `m4 -I. -I m4 -I sof -I common masterV41.m4 | alsatplg -c - -o masterV41.tplg` sans erreur, aucun widget orphelin. **Gate obligatoire** avant deploy.
2. **T0a Widget count** : `alsatplg -dump masterV41.tplg` → doit afficher 2 `W_PIPELINE` (créés par `pipe-dai*`), pas 4 (preuve que V4.1 n'ajoute pas de scheduler redondant). Total ~22 widgets : 2 PCMs + 8 buffers + 6 effets + 2 DAI + 2 `W_PIPELINE` + 2 schedulers `scheduling_plat`.
3. **T1 Load** : kernel charge sans -22 ni -110. Carte `softac5212tdm` + 2 PCMs.
4. **T2 Playback** : `aplay -D plughw:softac5212tdm,1 -c 8 -f S32_LE tone.wav`.
5. **T3 Capture** : `arecord -D plughw:softac5212tdm,0 -c 8 -f S32_LE -d 3 mics.wav`.
6. **T4 Full-duplex** : T2 + T3 simultanés 60 s, pas de xrun.
7. **T5 Bit-perfect** : stratégie T5a (loopback HW + compensation latence).
8. **T6 Contrôles** : `amixer cset name='1 Master Capture Volume' 32` change le gain. Les 2 bytes controls MBDR/DRC visibles.
9. **T7 Charge CPU** : lecture via `sof-logger` ou `debugfs /sys/kernel/debug/sof/etrace` des cycles `kcps` ou `cpc`.

7 Phase 1a.2 — stratégie tap HP_Monitor

L'investigation V4 a confirmé : **aucun exemple upstream** de `W_PCM_CAPTURE` comme sink d'un widget dans une pipeline `PLAYBACK`. Le framework `IPC3` ne l'interdit pas explicitement

(`host_legacy.c` détermine direction par widget type, pas par position dans la chaîne), mais le pattern est non testé.

Stratégie V4.1 retenue : option co-scheduled cross-pipeline same sched_comp.

- Pipeline playback (PIPE 6) reste avec effets OUT. Un `W_MUXDEMUX` inséré en fin alimente 2 sinks : B3 (vers DAI TX) et B4 (vers pipeline capture `HP_Monitor`).
 - Pipeline `HP_Monitor` (PIPE 7) : pipeline capture **piggyback scheduling** sur PIPE 6 via `PIPELINE_SCHED_COMP`. Le `sched_comp` partagé passe le guard `is_same_sched` (`pipeline-stream.c:505`).
 - DAPM cross-pipeline : `SectionGraph dapm(PIPELINE_SINK_7, N_MUXDEMUX_via_PIPELINE_DEMUX_6)`
 - en s'assurant que `PIPELINE_DEMUX_6` est bien exporté par `pipe-effects-playback-with-tap.m4` (bug V2.7 à ne pas reproduire).
- Fallback : loopback HW (câble TAC OUT → TAC IN) si le pattern cross-pipeline tap reste instable.

8 Questions pour l'investigation V4.1

1. **Gate compile V4.1** : le squelette `pipe-effects-capture.m4` (section 4.3) et `pipe-effects-playback.m4` (section 4.4) compilent-ils **ligne par ligne** via `m4 | alsatplg` ? Vérifier notamment :
 - `multiband_drc_coef_passthrough.m4` existe bien dans `sof/tools/topology/topology1/m4/` et produit un blob valide ?
 - `C_CONTROLMIXER` args 6 et 7 ("Channel register..." + `VOLUME_CHANNEL_MAP`) : l'argument 6 (Channel register and shift for Front Left/Right) est-il bien une **string libre non-quotée** comme dans le pattern upstream, ou faut-il le quoter/échapper ?
 - Le nom `kcontrol` référencé "`PIPELINE_ID Master Capture Volume`" – est-ce que `m4` expand bien `PIPELINE_ID` ici au moment de l'inclusion du fichier par `PIPELINE_PCM_ADD`, ou faut-il une autre syntaxe ?
2. **Pattern `V_PGA periods capture/playback`** : `capture = (2, DAI_PERIODS)`, `playback = (DAI_PERIODS, 2)`. C'est cohérent avec `pipe-volume-capture.m4:47` et `pipe-volume-playback.m4:33`. Mais est-ce que `W_MULTIBAND_DRC` et `W_DRC` en 8ch doivent aussi utiliser `DAI_PERIODS` du côté DAI, ou 2, 2 suffit ?
3. **Widget `W_PIPELINE` absent — confirmation** : le pattern upstream `pipe-drc-enabled-capture.m4`, `pipe-volume-capture.m4`, `pipe-multiband-drc-playback.m4` n'a **pas** de `W_PIPELINE`. Celui-ci est créé dans `pipe-dai-capture.m4:XX / pipe-dai-playback.m4:YY` via `DAI_ADD`. Confirmer en lisant ces deux fichiers que le `W_PIPELINE` créé utilise bien le même `PIPELINE_ID` que `PIPELINE_PCM_ADD` → un seul scheduler widget total.
4. **Co-scheduling Phase 1a.2 tap** : pour que le tap `HP_Monitor` fonctionne via `PIPELINE_SCHED_COMP_6` piggyback, quelle est la syntaxe exacte ? 13e arg de `PIPELINE_PCM_ADD` ? Y a-t-il des exemples upstream (regarder `sof-smart-amplifier.m4` ou `echo-ref`) ? Est-ce que la pipeline capture co-scheduled doit elle-même avoir `SCHEDULE_TIME_DOMAIN_DMA` ou hériter via le piggyback ?
5. **DRC pre-delay dans T5** : confirmer la valeur exacte de `CONFIG_DRC_MAX_PRE_DELAY_FRAMES` (256 ? autre ?) dans notre fork et que `drc_s32_default` applique **toujours** ce pre-delay même avec `enabled=0`. Existe-t-il une variante `drc_func` qui soit "enabled=0 + no pre-delay" ? Sinon, confirmer que la seule solution bit-perfect est de **retirer le widget DRC du graph** (stratégie T5b).
6. **MBDRC passthrough blob** : `multiband_drc_coef_passthrough.m4` existe-t-il dans notre branche `feature/sdma-ap2ap-phase1` ? Si oui, fournir le contenu (`num_bands`, `emp_deemp`, crossover coeffs). Si non, quel fork amont l'a (`sof main`, NXP, Intel) et faut-il le copier ?
7. **Red flags résiduels** : après toutes les corrections V4.1, reste-t-il des pièges non anticipés pour le premier build ? Notamment autour de :
 - `DAI_PERIODS` défini dynamiquement par `DAI_ADD` — le widget défini dans le fichier pipeline y a-t-il accès au moment de l'expansion `m4` ?

- `SCHEDULE_PERIOD = 1000 us` → 48 frames period → `COMP_PERIOD_FRAMES(PCM_MAX_RATE, SCHEDULE_PERIOD) = 48`, conforme aux buffer sizes mbdrc `buf_src[PLATFORM_MAX_CHANNELS]` ?
- Le nom du topology file `masterV41.m4` doit être ajouté à `CMakeLists.txt` pour que la compilation west/bitbake le prenne.

9 Livrables Phase 1a.1 MVP V4.1

- `sof/tools/topology/topology1/sof-imx8mp-tac5212-masterV41.m4`
- `sof/tools/topology/topology1/sof/sof-imx8mp-tac5212-pipe-effects-capture.m4`
- `sof/tools/topology/topology1/sof/sof-imx8mp-tac5212-pipe-effects-playback.m4`
- Patch `sof/app/boards/imx8mp-evk-mimx8ml8-adsp.conf` : `+CONFIG_COMP_MIXER=y`
- Patch `sof/tools/topology/topology1/CMakeLists.txt` : ajouter `masterV41.m4` à la liste
- `meta-local/recipes-kernel/linux/files/test-phase1a1-mvp-v41.sh` (tests T0–T7)
- Suppression des fichiers V2.7 buggés : `masterlite.m4`, `pipe-masterchain.m4`, `pipe-mastertap.m4`, `pipe-sai-merge.m4`, `mux_route_sai_merge.m4`, `demux_route_default.m4`

10 Protocole

- Aucune modification source SOF sans `critic_analyze` préalable (mode DOUX, fenêtre 2000 s post-analyze).
- Un changement = un commit = validation utilisateur.
- Gate `m4` | `alsatplg` OBLIGATOIRE avant tout deploy sur board.
- Si T0 échoue, retour à `critic_analyze` avec l'erreur exacte **avant** de modifier le squelette.
- Backup kernel+firmware baseline avant premier deploy (rescue).